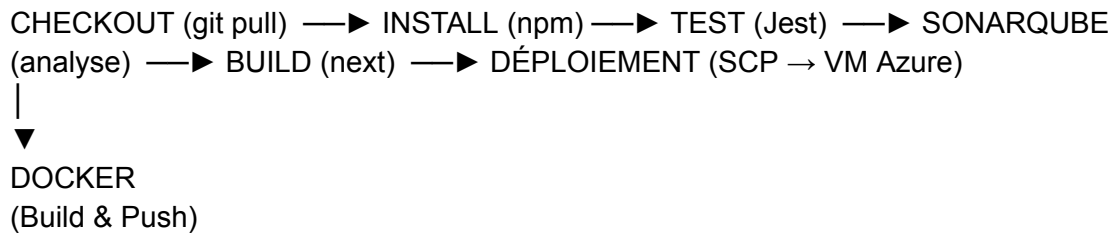


## Partie 1 : Architecture de la pipeline CI/CD

### 1.1 Schéma global



### 1.2 Enchaînement des étapes

- Checkout : Récupère le code source depuis GitHub (commande : checkout scm)
- Install : Installe les dépendances Node.js (commande : npm install)
- Test : Exécute 6 tests automatisés (unitaires, intégration, snapshot) avec Jest
- SonarQube : Analyse la qualité du code (bugs, vulnérabilités, couverture) via SonarCloud
- Build : Compile l'application Next.js pour la production (commande : npm run build)
- Docker : Construit l'image Docker et la publie sur Docker Hub
- Deploy : Déploie les fichiers buildés sur la VM Azure via SCP

### 1.3 Outils utilisés

- Jenkins (Docker local) : Orchestrateur CI/CD
- GitHub : Hébergement du code source
- Jest + React Testing Library : Tests automatisés
- SonarCloud : Analyse statique du code (SAST)
- Next.js 14 : Framework frontend React
- Docker + Docker Hub : Conteneurisation et registre d'images
- Azure VM (Ubuntu 24.04) : Serveur de déploiement

### 1.4 Types de tests

- Tests unitaires (3) : Validation de fonctions pures — formatIncidentStatus, getMarkerColor
- Tests d'intégration (2) : Rendu des composants React — Map, ProjectForm
- Tests de snapshot (1) : Détection de changements structurels — Page d'accueil

## Partie 2 : Runbooks

### Runbook 1 : Exécuter le projet en local

Prérequis :

- Node.js 18 ou supérieur
- Git
- Accès au dépôt GitHub

Étape 1 : Cloner le dépôt

```
git clone https://github.com/ShanthakumarThanus/CrisisView-Pipeline.git
cd CrisisView-Pipeline
```

#### **Étape 2 : Lancer le frontend**

```
bash
cd frontend
npm install --legacy-peer-deps
npm run dev
```

L'application est accessible sur : <http://localhost:3000>

#### **Étape 3 : Lancer le backend**

```
bash
cd backend
npm install
npm start
```

L'API est accessible sur : <http://localhost:3001>

## **Runbook 2 : Déployer via Docker Compose**

Prérequis :

- Docker installé
- Docker Compose installé

#### **Étape 1 : Lancer les conteneurs**

```
bash
cd CrisisView-Pipeline
docker-compose up -d
```

#### **Étape 2 : Vérifier le déploiement**

```
bash
docker ps
```

Deux conteneurs doivent apparaître :

- crisisview-frontend sur le port 3000
- crisisview-backend sur le port 3001

### Étape 3 : Accéder à l'application

- Frontend : <http://localhost:3000>
- Backend API : <http://localhost:3001>

## Runbook 3 : Relancer un pipeline Jenkins

### Prérequis :

- Jenkins démarré sur <http://localhost:8080>
- Job CrisisView-Pipeline configuré

### Étape 1 : Accéder à Jenkins

- URL : <http://localhost:8080>
- Identifiants : ceux configurés lors de l'installation

### Étape 2 : Lancer le build

1. Cliquer sur le job **CrisisView-Pipeline**
2. Cliquer **Build Now** (menu de gauche)
3. Surveiller la progression dans **Build History**
4. Cliquer sur le build en cours → **Console Output** pour voir les logs

### Étape 3 : Vérifier les résultats

- Dashboard SonarCloud :  
[https://sonarcloud.io/dashboard?id=ShanthakumarThanus\\_CrisisView-Pipeline](https://sonarcloud.io/dashboard?id=ShanthakumarThanus_CrisisView-Pipeline)
- Image Docker : <https://hub.docker.com/r/shanthakumarthanus/crisisview-frontend>

# Annexes

## A. Configuration Jenkins

| Credential ID    | Type            | Usage                               |
|------------------|-----------------|-------------------------------------|
| azure-vm-ip      | Secret text     | IP de la VM Azure                   |
| azure-vm-user    | Secret text     | Utilisateur SSH                     |
| azure-ssh-key    | SSH Private Key | Clé privée de connexion             |
| sonarcloud-token | Secret text     | Token d'authentification SonarCloud |
| docker-hub-pwd   | Secret text     | Token Docker Hub                    |

## B. Résultat des tests

Test Suites : 3 passed, 3 total

Tests : 6 passed, 6 total

Snapshots : 1 passed, 1 total

Couverture : 42.3% (Statements)

## C. Résultat SonarCloud

Quality Gate : Not computed

Open Issues : 0

Duplications : 0.0%

Coverage : 9.1%